LAB PERFORMANCE 14
Muhammad Waqas(240756)

```cpp
#include <iostream>
using namespace std;

// ================= AVL NODE =================
class Node {
public:
    int data;
    Node* left;
    Node* right;
    int height;

    Node(int val) {
        data = val;
        left = NULL;
        right = NULL;
        height = 1;
    }
};
// ================= HEIGHT FUNCTION =================
int getHeight(Node* n) {
    if (n == NULL)
        return 0;
    return n->height;
}

// ================= MAX FUNCTION =================
int max(int a, int b) {
    return (a > b) ? a : b;
}

// ================= BALANCE FACTOR =================
int getBalance(Node* n) {
    if (n == NULL)
        return 0;
    return getHeight(n->left) - getHeight(n->right);
}
```

```cpp
// ================= RIGHT ROTATION =================
Node* rightRotate(Node* y) {
    Node* x = y->left;
    Node* T2 = x->right;

    x->right = y;
    y->left = T2;

    y->height = max(getHeight(y->left), getHeight(y->right)) + 1;
    x->height = max(getHeight(x->left), getHeight(x->right)) + 1;

    return x;
}

// ================= LEFT ROTATION =================
Node* leftRotate(Node* x) {
    Node* y = x->right;
    Node* T2 = y->left;
    x->right = T2;

    x->height = max(getHeight(x->left), getHeight(x->right)) + 1;
    y->height = max(getHeight(y->left), getHeight(y->right)) + 1;

    return y;
}

// ================= INSERT FUNCTION =================
Node* insert(Node* node, int key) {

    // Normal BST insertion
    if (node == NULL)
        return new Node(key);

    if (key < node->data)
        node->left = insert(node->left, key);
    else if (key > node->data)
        node->right = insert(node->right, key);
```

```
    // Update height
    node->height = 1 + max(getHeight(node->left),
                           getHeight(node->right));

    // Get balance factor
    int balance = getBalance(node);

    // Left Left case
    if (balance > 1 && key < node->left->data)
        return rightRotate(node);

    // Right Right case
    if (balance < -1 && key > node->right->data)
        return leftRotate(node);

    // Left Right case
    if (balance > 1 && key > node->left->data) {
        node->left = leftRotate(node->left);
// Left Right case
if (balance > 1 && key > node->left->data) {
    node->left = leftRotate(node->left);
    return rightRotate(node);
}

// Right Left case
if (balance < -1 && key < node->right->data) {
    node->right = rightRotate(node->right);
    return leftRotate(node);
}

return node;
```

```cpp
// ================= INORDER TRAVERSAL =================
void inorder(Node* root) {
    if (root != NULL) {
        inorder(root->left);
        cout << root->data << " ";
        inorder(root->right);
    }
}

// ================= MAIN =================
int main() {
    Node* root = NULL;

    // Insert values mostly on right side
    // This makes right subtree deeper than left
    int values[] = {10, 20, 30, 40, 50, 60, 70, 80};
```

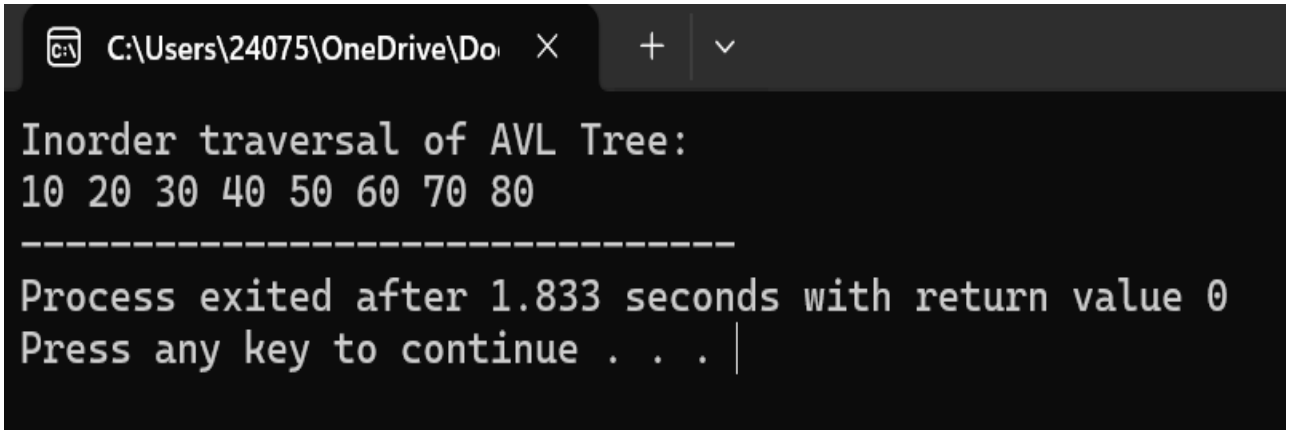
```cpp
int main() {
    Node* root = NULL;

    // Insert values mostly on right side
    // This makes right subtree deeper than left
    int values[] = {10, 20, 30, 40, 50, 60, 70, 80};

    for (int i = 0; i < 8; i++) {
        root = insert(root, values[i]);
    }

    cout << "Inorder traversal of AVL Tree:\n";
    inorder(root);

    return 0;
}
```


C:\Users\24075\OneDrive\Do
Inorder traversal of AVL Tree:
10 20 30 40 50 60 70 80

Process exited after 1.833 seconds with return value 0
Press any key to continue . . .